

Homework 4: Generative Modeling CPE 487/587

Instructor: Rahul Bhadani

Due: April 24, 2026, 11:59 PM
100 Points

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

Upload a .pdf on Canvas with the format CPE487587-LastFirst-HW-XX. For example, if your name is Sam Wells, your file name should be CPE487587-LastFirst-HW-XX.pdf. If there is a programming assignment, then you should include your source code along with your PDF files in a zip file CPE487587-LastFirst-HW-XX.zip. Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

All CPE 587 students are required to submit their response in a Latex-generated PDF.

Provide a Github commit hash and GitHub repo URL for this assignment that supercedes the above instruction. In addition, include any plots in the PDF for practice portion if asked.

1 Image Quality Comparison from Generative Models

The goal is to make comparative assessment of image quality of three generative models: (i) Variational AutoEncoder; (ii) Generative Adversarial Network; (iii) Diffusion Model.

We will use Celeb-FaceA dataset for benchmarking our comparison.

Celeb Face A dataset is available on Lovelace machine at location

/data/CPE_487-587/img_align_celeba.zip.

Your job is broken into following steps:

1. Create three generative AI models (i) Variational AutoEncoder; (ii) Generative Adversarial Network; (iii) Diffusion Model using Cele-FaceA dataset. Save the trained models as onnx files.
2. Load trained models and generate 25 random samples from trained models.
3. Assess their quality on several metrics (defined below) and provide a comparative visualization of your choice (bar plot, box plot ,etc.).
4. Commit your code to GitHub.

1.1 Image Sharpness Metrics

Let $I : \Omega \rightarrow [0, 1]$ be a grayscale image defined on a discrete pixel domain $\Omega = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ with width W and height H . The total number of pixels is $N = WH$.

For continuous-domain analogues, $I(x, y)$ denotes a scalar intensity function on a bounded region $\Omega \subset \mathbb{R}^2$.

Definition 1.1 (Image gradient). The spatial gradient of I at pixel (x, y) is the vector

$$\nabla I(x, y) = \begin{pmatrix} I_x(x, y) \\ I_y(x, y) \end{pmatrix},$$

where I_x and I_y denote partial derivatives (or their finite-difference approximations) along the horizontal and vertical directions, respectively.

Definition 1.2 (Laplacian operator). The scalar Laplacian of I is

$$\nabla^2 I(x, y) = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}.$$

In discrete form, the most common 3×3 kernel is

$$\mathbf{L} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix},$$

so the discrete Laplacian response at pixel (x, y) is the convolution $L(x, y) = (I * \mathbf{L})(x, y)$.

1.1.1 Variance of the Laplacian

Definition 1.3 (Variance of Laplacian , VoL). Let $L(x, y) = (\nabla^2 I)(x, y)$ be the discrete Laplacian response. Define the sample mean

$$\bar{L} = \frac{1}{N} \sum_{(x,y) \in \Omega} L(x, y).$$

The *Variance of Laplacian* focus measure is

$$F_{\text{VoL}} = \frac{1}{N} \sum_{(x,y) \in \Omega} (L(x, y) - \bar{L})^2.$$

Remark 1.1. Because the Laplacian acts as a second-order high-pass filter, its response is large at edges and fine-textured regions and near zero in smooth regions. Sharper images produce higher-variance Laplacian maps; blurred images drive $F_{\text{VoL}} \rightarrow 0$.

1.1.2 Tenengrad Criterion

Definition 1.4 (Sobel gradient components). The horizontal and vertical Sobel operators of size 3×3 convolve the image with the kernels

$$\mathbf{S}_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{S}_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix},$$

yielding $G_x(x, y) = (I * \mathbf{S}_x)(x, y)$ and $G_y(x, y) = (I * \mathbf{S}_y)(x, y)$.

Definition 1.5 (Gradient magnitude). The pointwise gradient magnitude is

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}.$$

Definition 1.6 (Tenengrad criterion , TEN).

$$F_{\text{TEN}} = \frac{1}{N} \sum_{(x,y) \in \Omega} (M(x, y) - \bar{M})^2,$$

where $\bar{M} = \frac{1}{N} \sum_{(x,y)} M(x,y)$ is the mean gradient magnitude. Equivalently, the sum-of-squares variant

$$F_{\text{TEN}}^{(\Sigma)} = \sum_{(x,y) \in \Omega} M(x,y)^2$$

is also widely used.

Remark 1.2. F_{TEN} is a first-order analogue of F_{VOL} : it measures the strength and spread of edge responses rather than second-order curvature. For a diffraction-limited optical system it has been shown to be a reliable autofocus criterion.

1.1.3 Frequency-Domain High-Frequency Energy Ratio

Definition 1.7 (Discrete 2-D Fourier Transform). The 2-D Discrete Fourier Transform (DFT) of I is

$$\hat{I}(u,v) = \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} I(x,y) e^{-2\pi i \left(\frac{ux}{W} + \frac{vy}{H} \right)}, \quad u \in \{0, \dots, W-1\}, v \in \{0, \dots, H-1\}.$$

After applying the *frequency shift* (centering the zero-frequency component), the shifted spectrum is $\tilde{I}(u,v)$ with $(u,v) = (0,0)$ corresponding to the DC component at the array centre.

Definition 1.8 (Low-frequency disc mask). Let $r = \lfloor \alpha \cdot \min(W, H) \rfloor$ for a bandwidth parameter $\alpha \in (0, \frac{1}{2})$. Define the low-frequency disc

$$\mathcal{D}_r = \left\{ (u,v) \mid \left(u - \frac{W}{2}\right)^2 + \left(v - \frac{H}{2}\right)^2 \leq r^2 \right\},$$

and its complement $\mathcal{D}_r^c = \Omega \setminus \mathcal{D}_r$, which forms the high-frequency annular region.

Definition 1.9 (High-frequency energy ratio , HFE).

$$F_{\text{HFE}} = \frac{\sum_{(u,v) \in \mathcal{D}_r^c} |\tilde{I}(u,v)|}{\sum_{(u,v) \in \Omega} |\tilde{I}(u,v)|},$$

with the convention $F_{\text{HFE}} = 0$ when the total energy vanishes.

Remark 1.3. By Parseval's theorem, $\sum_{(u,v)} |\hat{I}(u,v)|^2 = N \sum_{(x,y)} |I(x,y)|^2$, so an analogous energy-squared variant replaces $|\tilde{I}|$ with $|\tilde{I}|^2$. Sharper images concentrate more energy in $\mathcal{D}_r^c$ and therefore yield larger F_{HFE} .

1.1.4 Mean Local Standard Deviation

Definition 1.10 (Local mean and variance via uniform filter). For a window of size $w \times w$, define the *box-filtered* versions of I and I^2 :

$$\mu_w(x, y) = \frac{1}{w^2} \sum_{s=-\lfloor w/2 \rfloor}^{\lfloor w/2 \rfloor} \sum_{t=-\lfloor w/2 \rfloor}^{\lfloor w/2 \rfloor} I(x + s, y + t),$$

$$\mu_{w^2}(x, y) = \frac{1}{w^2} \sum_{s, t} I(x + s, y + t)^2.$$

The local variance is given by the *König–Huygens identity*:

$$\sigma_w^2(x, y) = \max(0, \mu_{w^2}(x, y) - \mu_w(x, y)^2),$$

and the local standard deviation is $\sigma_w(x, y) = \sqrt{\sigma_w^2(x, y)}$.

Definition 1.11 (Mean local standard deviation, MLSD).

$$F_{\text{MLSD}} = \frac{1}{N} \sum_{(x, y) \in \Omega} \sigma_w(x, y).$$

Remark 1.4. F_{MLSD} is a direct texture measure: regions with rapid intensity variation (fine detail, high-frequency texture) contribute large σ_w , whereas uniform or blurred regions contribute near zero. The $\max(0, \cdot)$ guard prevents numerical artefacts from floating-point cancellation.

1.1.5 GLCM Contrast

Definition 1.12 (Quantised image). Given Q grey levels, the quantised image is

$$I_Q(x, y) = \lfloor I(x, y) \cdot (Q - 1) \rfloor \in \{0, 1, \dots, Q - 1\}.$$

Definition 1.13 (Gray-Level Co-occurrence Matrix). For a displacement vector $\mathbf{d} = (d, \theta)$ (distance d and angle θ), let $\Delta x = d \cos \theta$, $\Delta y = d \sin \theta$. The unnormalised GLCM entry counting co-occurrences of intensity pair (i, j) is

$$C_{ij} = \# \left\{ (x, y) \in \Omega \mid I_Q(x, y) = i, I_Q(x + \Delta x, y + \Delta y) = j \right\}.$$

The *normalised* GLCM is

$$P_{ij} = \frac{C_{ij}}{\sum_{i'=0}^{Q-1} \sum_{j'=0}^{Q-1} C_{i'j'}}, \quad \sum_{i, j} P_{ij} = 1.$$

Symmetrisation yields $P_{ij} \leftarrow (P_{ij} + P_{ji})/2$.

Definition 1.14 (GLCM Contrast).

$$F_{\text{CON}} = \sum_{i=0}^{Q-1} \sum_{j=0}^{Q-1} (i - j)^2 P_{ij}.$$

Remark 1.5. The weight $(i - j)^2$ penalises co-occurrences between dissimilar grey levels at the displacement $\mathbf{d}$. Images with sharp edges or fine texture have many large-gap neighbours, yielding high F_{CON} . Blurred images concentrate mass near the diagonal ($i \approx j$), driving $F_{\text{CON}} \rightarrow 0$.

Remark 1.6. Related GLCM second-order statistics include:

$$F_{\text{ENE}} = \sum_{i,j} P_{ij}^2 \quad (\text{Energy / Angular Second Moment}),$$

$$F_{\text{HOM}} = \sum_{i,j} \frac{P_{ij}}{1 + (i - j)^2} \quad (\text{Homogeneity / Inverse Difference Moment}),$$

$$F_{\text{ENT}} = - \sum_{i,j} P_{ij} \log P_{ij} \quad (\text{Entropy}).$$

1.2 Reading The Image Dataset

You may use the following code to read the image dataset:

```

1  import torch, torchvision
2
3  from torch.utils.data import DataLoader
4  from torchvision import datasets, transforms
5
6
7  import torch
8  from torch.utils.data import Dataset, DataLoader
9  from torchvision import transforms
10 from PIL import Image
11 import zipfile
12 import io
13
14 class CelebAZipDataset(Dataset):
15     def __init__(self, zip_path, transform=None):
16         self.zip_path = zip_path
17         self.transform = transform
18
19         # Open zip once to collect all image filenames
20         with zipfile.ZipFile(zip_path, 'r') as zf:
21             self.image_names = sorted([
22                 name for name in zf.namelist()
23                 if name.lower().endswith(('.jpg', '.jpeg', '.png'))
24             ])
25
26     def __len__(self):
27         return len(self.image_names)
28
29     def __getitem__(self, idx):
30         # Re-open zip per worker (required for DataLoader multiprocessing)
31         with zipfile.ZipFile(self.zip_path, 'r') as zf:
32             with zf.open(self.image_names[idx]) as f:

```

```

33         img = Image.open(io.BytesIO(f.read())).convert('RGB')
34
35         if self.transform:
36             img = self.transform(img)
37
38         return img
39
40
41
42     transform = transforms.Compose([
43         transforms.Resize(64),
44         transforms.CenterCrop(64),
45         transforms.ToTensor(),
46         transforms.Normalize([0.5]*3, [0.5]*3) # to [-1, 1]
47     ])
48
49     dataset = CelebAZipDataset(
50         zip_path='/data/CPE_487-587/img_align_celeba.zip',
51         transform=transform
52     )
53
54     dataloader = DataLoader(
55         dataset,
56         batch_size=128,
57         shuffle=True,
58         num_workers=4,
59         pin_memory=True
60     )
61
62     batch = next(iter(dataloader))
63     print(f"Batch shape: {batch.shape}") # torch.Size([128, 3, 64, 64])
64     print(f"Value range: [{batch.min():.2f}, {batch.max():.2f}]") # [-1.0, 1.0]
65

```

1.3 Deliverables

1. In deeppl subpackage, provide gen_model.py module that contains implementation of VAE, GAN, and Diffusion Model.
2. You should also provide an implementation for GenModelTrainer class that should be agnostic to model of choice, be it a VAE, GAN or Diffusion Model.
3. Your trainer should be able to save an intermediate ONNX file every x epoch where you should define x in the runtime at command line.
4. You should be able to write your script in such a way that it should take command line argument such as number of epochs to train, ratio of training dataset and validation dataset to use for training, as well which model to train with, such as VAE, GAN, Diffusion Model.
5. You should train on all three models and save the ONNX file.
6. You should provide an impl file in scripts folder that imports model classes and the trainer class that on execution trains the model and saves them.
7. You should provide another impl file in scripts folder that loads the saved model to perform inference and generate 25 synthetic images and perform evaluation on the metrics defined

above in this document. You may choose to provide implementation for metrics in your package in a suitable subpackage and import in the impl file. Evaluate generate images for the metrics and provide an aggregated visualization plot.

8. Write a helping bash script `genmodel_impl.sh` to orchestrate training and inference.
9. Create a README file the provides instructions on how to run your code.